System Design Simulator: Full Plan

Oct 13, 2026 · @Iron Bunny

A browser-based simulator where engineers draw an architecture, set a workload, and see where it saturates, fails or costs too much, using calibrated real-world numbers.

Vision and scope

The simulator is a capacity-planning tool: users model an architecture, apply a workload, and learn where it saturates, fails or costs too much before anything is built.

- **Users:** engineers and architects sizing and validating distributed systems.
- **Core promise:** results come from calibrated real-world numbers (service times, pool sizes, throughput limits), not from guesses.
- **Two source specs:** the Enterprise System Design Simulator blueprint (product shell, canvas, node types, fault injection) and SysLab Architect (graph and matrix model, fluid-flow versus discrete-event modes, invariants). This plan merges them.
- **Assumption:** "real-base" is read as "grounded in real benchmark data". Confirm or correct this in the open decisions.

Out of scope for v1: exporting Terraform or CloudFormation, multi-region network modeling, and team collaboration features.

Competitive landscape

Our target is to beat four existing simulators by combining what each does best with something none of their pages show: simulation numbers that come from cited, calibrated data. Findings below come from screenshots you supplied on 14 Oct 2026, so they show one screen of each product and nothing I could click or run.

Simulator	Learn from it	What the screenshot shows	Gap we can exceed
SyDe (v1.8)	More detailed components	Three stages (Design, Dev Guided, Prod Sim) plus Wiki and Community tabs; palette grouped by category (Client-Side 10 types, Infrastructure 7); Flow View filter for Read, Write and Async traffic; Lanes view; Security layer with a posture score (36/100, high risk, 4 modeled threats, 24 controls); quick-start templates (E-Commerce, AI RAG, Payments, Secure Multi-Site); import from Terraform, Kubernetes, JSON, XML or Mermaid; step-by-step design analysis guide	The config panel was not open, so per-component parameter depth is unconfirmed; several palette entries share one generic icon; no run metrics visible
SysSimulator	Clean UI	Dark canvas; each node shows live rps, p99 and a utilization bar; edges labeled by protocol (HTTPS, SQL queries, Load balanced) with animated flow; top bar with RPS, latency, error rate and bottleneck count; Simulate, Chaos, Cost, Metrics and Blueprints buttons; speed and traffic controls; AI-era components (Agent Runtime, MCP Tool Server, Vector Store, Tool Registry)	Top bar says 100 RPS but the client node shows 20 rps, and I cannot reconcile the figures from one frame; no source shown for latency numbers

Simulator	Learn from it	What the screenshot shows	Gap we can exceed
SysLab	Tutorial guideline	Scenario workspace ("Design a File Storage Service") with Brief, Practice Mode, a countdown timer (119m left), Evaluate, AI Mentor, plus Scenarios and Learn tabs; each node shows utilization and estimated cost per hour; results panel with a health score (56), RPS, average, p50 and p95 latency, a 60-second throughput and latency chart, and a bottleneck list with advice (a worker at 87% utilization; "queueing delay starts compounding past roughly 75%")	The 75% threshold looks like a rule of thumb with no source or calibration shown; the auto-laid-out canvas is crowded with crossing edges
SystemLore	Game-like framework	Endless "Keep it alive" mode with a HUD (RPS, P95 against a 300 ms target, error %, cache hit %, cash, running cost per second, score); random incidents such as viral traffic with response choices (add app servers, rate-limit everyone, cache the hot pages, serve a static fallback); a shop bar pricing each component ($17 to $40); animated request particles	Abstract circles and game money rather than an engineering model; no per-class traffic

What we take from each:

ID	Requirement	Learned from	Priority
C1	Clean canvas: live rps, p99 and utilization on every node, protocol-labeled animated edges, one summary bar (RPS, latency, errors, bottlenecks)	SysSimulator	P0
C2	Scenario-based tutorials: brief, hints, an evaluate step, an optional timer, and a mentor that explains each result	SysLab	P1
C3	Challenge mode: incident events with response choices, a running cost, a score and a post-mortem	SystemLore	P2
C4	Deep component models: every behavior-changing parameter configurable and explained in a config panel; palette grouped by category	SyDe	P0
C5	Blueprint and quick-start template library, each with its calibration source	SysSimulator, SyDe	P1
C6	Bottleneck explainer: for each bottleneck show the cause, the numbers (utilization, queue length, expected wait from the queueing model) and a suggested fix, derived from the model rather than a flat threshold	SysLab	P0
C7	Canvas filter by request class (read, write, async)	SyDe	P1
C8	Import a topology from Terraform, Kubernetes, Mermaid or JSON (import only; export stays out of v1)	SyDe	P2

Where we exceed all four: every number shows its source and confidence (calibration library); edges carry typed requests instead of one traffic number; two run modes on one topology; deterministic replay and flow-conservation checks; fidelity measured against real load tests. C1 to C8 are merged into the requirements table: C1 into F7, C4 into F3, and C2, C3, C5, C6, C7, C8 as F20, F21, F22, F23, F24, F25.

Sources: SysSimulator, SystemLore, SyDe, SysLab (page not readable). Screenshots of all four simulators were supplied by you on 14 Oct 2026.

Multi-cloud: one engine, provider catalogs as data

The simulator is cloud-agnostic: one simulation engine for distributed-system behavior, with AWS, Azure, GCP, Kubernetes and generic components added as catalogs of data and capability adapters. AWS is one supported environment, never the core model.

Generic core and catalogs. Every component is defined once in a canonical, provider-neutral model (concurrency or server count, service time per request class, queue capacity, throughput limit, scaling policy, failure settings, payload size, cost inputs). A catalog entry maps a real service onto that model and keeps its own constraints (SKU, region, quotas, scaling limits, consistency and durability behavior). Services from different providers are never treated as exact equivalents, and unknown fields are shown as unknown.

Catalog and calibration stay separate. A catalog entry says what a service can be configured to do. A calibration profile says how it measured: source URL or dataset, capture date, software or service version, region and tier, request class, configuration, and a label of Measured, Vendor figure or Estimated. A catalog entry is never proof of a latency or throughput value, and no benchmark, price or quota is invented. Provider comparisons use the same workload and topology shape, with capacity, tiers, quotas and costs set from sourced inputs.

Both run modes use the same canonical behavior, even when a service exposes only part of it. The first validation target is one reference architecture built from generic components, then mapped to AWS, Azure and GCP.

The requirements for this section are F15 to F19 in the Requirements table.

Decisions so far

- Default request classes are Read, Write, Async event and Bulk, and users can add custom classes.
- Calibration is a full page, not a side panel. It is where every number's source, date and confidence is kept.
- **T**he provider is chosen per component, not per project. The Workspace palette lists provider services directly (AWS, Azure, GCP, Kubernetes) plus a Generic group, so one design can mix providers. The Catalog page is for browsing catalogs and comparing providers; a design does not have to start there.

Requirements

The tool must let a user build a topology, run it under load, and trust the answer; P0 is the minimum for a usable first version. This is the single list of requirements. IDs are stable: F for functional, N for non-functional. Source says where each one came from; "Proposed" and "Open" items still need your confirmation.

ID	Requirement	Area	Priority	Source
F1	Node-link canvas: drag, connect, pan, zoom, grid snap	Canvas	P0	Blueprint
F2	Component palette: provider services listed directly under AWS, Azure, GCP and Kubernetes, plus a Generic group (clients, load balancer, service, cache, database, queue, worker pool, CDN, object store). The sidebar works like a file explorer: provider folders that expand, category headings, and two view modes, a button view (icon above, label below) and a list view; each service shows whether its limits are checked	Canvas	P0	Blueprint, SyDe
F3	Per-component configuration: concurrency, pool size, service time per request class, capacity. Every behavior-changing parameter is configurable and explained in the panel	Model	P0	Blueprint, SyDe
F4	Typed edges: request classes, rates, payload sizes, call semantics (see System model)	Model	P0	Proposed
F5	Workload definition: constant, Poisson, step and spike profiles, per request class	Run	P0	Blueprint
F6	Two run modes (fluid-flow and discrete-event) on the same topology	Run	P0	SysLab Architect spec
F7	Live metrics: P50/P90/P99 latency, queue depth, utilization, loss, bottleneck heatmap, shown on every node with labeled edges and one summary bar (rps, latency, errors, bottlenecks)	Results	P0	Both specs, SysSimulator
F8	Fault injection: node crash, latency spike, link cut, packet drop	Run	P1	Blueprint
F9	Blast-radius view: components impacted by a failed node	Results	P1	SysLab Architect spec

ID	Requirement	Area	Priority	Source
F10	Calibration library as a full page: a profile per component with source, date, version, tier, region and confidence (Measured, Vendor figure or Estimated); load-test CSV import	Calibration	P0	Proposed
F11	Cost estimate from instance and pricing inputs	Providers	P2	Proposed
F12	Save, load and share topologies	Platform	P1	Blueprint
F13	Nested sub-systems (cluster as one node)	Platform	P2	Blueprint
F14	Export to IaC and API specs	Platform	P3	Blueprint, deferred
F15	Canonical provider-neutral component schema, with unknown fields shown explicitly	Providers	P0	Multi-cloud brief
F16	Generic catalog as the default palette; the engine contains no provider-specific code	Providers	P0	Multi-cloud brief
F17	Provider catalogs (AWS, Azure, GCP, Kubernetes) as data and capability adapters that keep each service's own constraints. The provider is chosen per component in the Workspace, so one design can mix providers; each component is a provider service with a configuration (instance class, count, region, scaling), and the generic kind is its type	Providers	P1	Multi-cloud brief
F18	Same topology and workload compared across providers, with source and confidence beside each key value	Providers	P1	Multi-cloud brief
F19	Edge semantics defined before build: routing versus fan-out, offered/admitted/completed/dropped/retried counts, bandwidth, retry budget, circuit breaker	Model	P0	Multi-cloud brief
F20	Scenario-based tutorials: brief, hints, an evaluate step, an optional timer, and a mentor that explains each result	Learning	P1	SysLab
F21	Challenge mode: incident events with response choices, a running cost, a score and a post-mortem	Learning	P2	SystemLore
F22	Blueprint and quick-start template library, each with its calibration source	Learning	P1	SysSimulator, SyDe

ID	Requirement	Area	Priority	Source
F23	Bottleneck explainer: cause, numbers (utilization, queue length, expected wait from the queueing model) and a suggested fix, derived from the model rather than a flat threshold	Results	P0	SysLab
F24	Canvas filter by request class (read, write, async, bulk)	Canvas	P1	SyDe
F25	Import a topology from Terraform, Kubernetes, Mermaid or JSON (import only; export stays out of v1)	Platform	P2	SyDe
N1	Fidelity: simulated bottleneck predictions are checked against actual load-test results. The Blueprint's numeric target (Simulation Fidelity Ratio, over 92% correlation with staging load-test benchmarks) is dropped for now. Open KPI until the metric, dataset and method are defined	Quality	P0	Blueprint KPI
N2	Deterministic replay: same topology version, workload, mode, seed and calibration give an identical timeline	Quality	P0	SysLab Architect spec
N3	Conservation of flow holds at every tick (inflow = outflow + change in backlog), with retries and drops defined	Quality	P0	SysLab Architect spec
N4	UI stays responsive: simulation off the main thread, 60 FPS canvas	Quality	P0	SysLab Architect spec
N5	Time to build and evaluate a design under 4 hours	Quality	P1	Blueprint KPI
N6	Limits: maximum nodes per canvas, simulated duration, browser support, discrete-event request volume. Numbers not yet set	Quality	P1	Open
N7	Mode agreement: stable, light-load scenarios match within a stated tolerance; scenarios where the modes should differ are documented	Quality	P0	Multi-cloud brief
F26	Every catalog value carries a limit type: hard limit, adjustable quota, best-effort target or performance guidance. Only the first two may act as a hard ceiling in the engine	Calibration	P0 (proposed)	AWS check, Oct 18, 2026

ID	Requirement	Area	Priority	Source
F27	Every catalog value stores unit, scope, conditions (API type, engine, execution model), source URL, retrieval date and status. Provider entries are split where behavior differs (API Gateway types, SQS Standard and FIFO, ECS and EKS configurations, MemoryDB as a database profile)	Calibration, Catalog	P0 (proposed)	AWS check, Oct 18, 2026; Provider catalogs, Catalog value rules
F28	Every calibration value can be typed by hand: median and P95 service time per request class, connection pool and other capacity inputs, plus custom request classes. The label follows the evidence: no source is Estimated, a source link is Vendor figure, an imported load-test CSV is Measured. P95 must be at least the median; Reset to default restores the starting values. The CSV is optional	Calibration	P0 (proposed)	Your request, Oct 19, 2026; Calibration screen
F29	Mixed-provider designs: each component carries its own provider, so one topology can use AWS, Azure and GCP together. A connection that leaves a provider is marked, and its latency and capacity are Estimated until measured	Canvas, Providers	P0 (proposed)	Your request, Oct 19, 2026; Workspace screen

System model

Terms used from here on: **offered** (arrives at a component or edge), **admitted** (accepted into service or the queue), **completed** (finished successfully), **dropped** (lost or rejected), **retried** (a repeat attempt after a failure). Each is a separate count.

Every component on the canvas is one canonical component with an optional provider binding. The engine reads only the canonical part. The provider binding is data for the UI, for cost, and for checking limits.

A component has four parts:

1. **Identity:** an id, a name and a canonical kind (traffic generator, load balancer, gateway, service, worker, queue, event bus, cache, relational database, NoSQL database, object storage, CDN).
2. **Behavior (shared across providers):** the fields the engine uses.
3. **Provider binding (optional):** which real service this stands for and its own constraints.

4. **Calibration references:** pointers to profiles that supply measured values.

Behavior field	Meaning	Notes
`servers`	Concurrency: parallel workers, slots, connections	c in the queueing model
`serviceTime[class]`	Time to serve one request of a class, as a distribution	Mean for fluid mode, sampled in discrete-event mode
`queueCapacity`	Waiting room size; beyond it, requests are rejected	Finite or unbounded
`throughputLimit`	Hard cap on requests or bytes per second, if the service has one	Optional
`scaling`	Fixed, or autoscaled with minimum, maximum and delay	Delay matters during spikes
`availability`	Failure behavior: crash rate, recovery time	Used by fault injection
`payload[class]`	Request and response bytes per class	Feeds bandwidth
`cost`	Unit prices and billing basis	Only from sourced inputs

Every value is one of three states: **set** (with a source and a confidence label), **unknown**, or **not applicable**. An unknown field is shown as unknown on screen and blocks a run only if the engine truly needs it; the user can then enter an estimate, which is labeled Estimated.

Provider binding holds what only that service has: provider, service name, SKU or instance family, region, pricing model, quotas, scaling limits, durability and consistency notes, and links to the provider's documentation with the date read. These are never merged into the behavior fields by guess.

A catalog entry is data with three parts: the canonical kind it maps to, the list of behavior fields the service supports (it may support only a subset), and the schema of its provider-specific fields. An **adapter** turns a user's choices for that service (for example a SKU and a count) into canonical behavior fields, and reports which fields it could not fill.

The catalog says what a service can be set to do. A calibration profile (see Calibration) says how it measured. Neither stands in for the other.

Choosing a component takes three levels. A provider does not offer one service per category, and the simulator must not pretend it does. The palette lists services directly, so the user starts at level 2 and the kind follows from the service.

Level	The user chooses	Example
1. Kind	The generic behavior to model	Database
2. Service	The real implementation, from the chosen provider's catalog	Relational (RDS, Aurora) or NoSQL (DynamoDB)
3. Configuration	Capacity and operating details of that service	Instance class, count, region, scaling

The palette lists provider services directly, so placing one fixes its kind. The inspector then asks for the configuration. Different services of one kind expose different settings: a serverless function shows concurrency and duration limits where a container service shows instance size and count. Their simulated behavior differs accordingly, and services of one kind are never treated as interchangeable.

Shared edge contract

An edge is defined once, statically. Each run mode then derives its own runtime state from it (see Simulation engine). The contract has six groups.

Group	Fields	Rule
Endpoints	`from`, `to`	Directed. A pair may have several edges
Routing	`mode` (single, weighted, broadcast, conditional), `weight`, `condition`, `fanOut`	Defined below
Classes	per class: `active`, `requestBytes`, `responseBytes`, `cacheable`	A class not routed on an edge carries zero flow there
Network	`latencyMs`, `bandwidthBps`, `bandwidthApplies` (request, response, both), `lossProb`, `serialization` (on or off)	Loss is a network drop (see C4)

Group	Fields	Rule
Call	`kind` (sync or async), `timeoutMs`, `deadlineMs`	Async calls do not wait for the response
Resilience	`retry` {max, backoff, baseMs, jitter, budget, onlyIdempotent}, `breaker` {errorThreshold, windowMs, openMs, probes}	Retries and breaker states are counted separately

Routing and fan-out, defined. Let a component admit rate λ of a class. An outgoing edge e then receives the offered rate below. Fan-out `m_e` is the number of calls made on edge e per request that takes that edge; it is not a probability, and the default is 1.

```
\text{single: } \lambda_e = m_e\,\lambda \qquad \text{weighted: } \lambda_e = \frac{w_e}{\sum_{j\in G} w_j}\, m_e\,\lambda \qquad \text{broadcast: } \lambda_e = m_e\,\lambda \text{ for every edge}
```

In weighted routing the weights are normalized across the edges of the same group G, so they do not have to sum to 1. In broadcast every edge gets the full rate. Conditional routing sends a class (v1: by request class only) to an edge, and the fraction is the share of that class.

Worked examples (illustrative numbers, not benchmarks)

Case	Setup	Result
Weighted	App service sends 800 rps of reads to a primary database (weight 1) and a replica (weight 3)	Primary 200 rps, replica 600 rps
Broadcast	Event bus admits 150 events per second and broadcasts to an email worker and an analytics worker	150 per second to each, 300 offered downstream in total
Fan-out	App service calls an auth service twice per request (`m` = 2) at 200 rps of writes	400 calls per second to auth
Retries	Failure chance per attempt f = 0.1, up to 2 retries	Attempts per request = 1 + 0.1 + 0.01 = 1.11, so offered load is 11% higher

Case	Setup	Result
Bandwidth	400 rps, 2 KB request, 20 KB response, 100 Mbit/s link	Request side 6.4 Mbit/s; response side 64 Mbit/s (64% of the link); serialization of one response 1.6 ms
Cache	800 rps of reads, hit rate 0.85 at the cache	680 rps answered by the cache; 120 rps continue to the database

The retry factor is the sum of f to the power i, for i from 0 to the retry limit r:

```
A = \sum_{i=0}^{r} f^{i} = \frac{1-f^{\,r+1}}{1-f}
```

A retry budget caps retries at a fraction of first attempts, so a failing dependency cannot be flooded by its own callers. The budget value, and the exact circuit-breaker transitions, are decisions to confirm before they are asserted in tests (see Validation and tests).

Provider catalogs

A catalog lists provider services directly, grouped by category. Each service maps onto one canonical kind and fills the shared behavior fields it can; anything else stays in its provider fields. The lists below come from the providers' official product pages (found Oct 19, 2026). Each service carries one of four statuses: Verified, Partial, Listed (not checked) or No limits on page (see Components by provider). Every entry stores the official page and the date read (Oct 20, 2026) before it ships.

Canonical kind	AWS	Azure	GCP	Kubernetes / self-hosted	Check per service before shipping
Gateway	API Gateway	API Management	API Gateway, Apigee	Ingress or gateway controller	Rate limits, timeouts, payload limits
Load balancer	ALB, NLB	Load Balancer, Application Gateway	Cloud Load Balancing	Service, ingress	Layer, connection limits, scaling behavior
Service	EC2, ECS, EKS	AKS, Container Apps	GKE, Cloud Run	Deployment	Concurrency per instance, scaling delay, instance sizes

Canonical kind	AWS	Azure	GCP	Kubernetes / self-hosted	Check per service before shipping
Function	Lambda	Functions	Cloud Run functions	Function runtime	Concurrency, cold starts, duration limits
Queue	SQS	Service Bus	Pub/Sub	Broker deployment	Ordering, delivery guarantee, size and retention limits
Event bus (publish and subscribe, routing)	SNS, EventBridge	Event Grid	Pub/Sub	Broker deployment	Fan-out limits, delivery semantics
Cache	ElastiCache, MemoryDB	Cache for Redis	Memorystore	Cache deployment	Node sizes, eviction, replication, durability
Relational database	RDS, Aurora	Azure SQL	Cloud SQL, Spanner	Database deployment	Connection limits, replicas, failover
NoSQL database	DynamoDB	Cosmos DB	Firestore	Database deployment	Partition and throughput limits, consistency
Object storage	S3	Blob Storage	Cloud Storage	Object store	Request-rate limits, consistency
CDN	CloudFront	Front Door, CDN	Cloud CDN	CDN or cache proxy	Cache behavior, origin limits

What the mapping does. Each row's services all map onto the same canonical kind and fill the shared behavior fields they can. Anything a service has that the model lacks (for example a partition limit or a throughput unit) goes in its provider fields and is used for limit warnings and cost, not guessed into the engine.

Limits of the mapping

- Services in a row are alike for simulation, not equal. A Lambda function and an ECS service are both "compute" to the engine; their scaling and billing differ.
- A service may support only some behavior fields. Unsupported fields show as not applicable; unknown ones show as unknown.
- Managed services can hide their internals. Where a value cannot be known, we ask for a load test, not an assumption.

- Pricing and quotas change. Each is stored with a source link and a date and is flagged when old.

Block and file storage that attaches to compute (for example EBS or EFS) is modeled in v1 as a property of the database or service that uses it, not as a separate node. Object storage and CDN stay separate nodes because they serve requests on their own.

Components by provider

The Workspace palette and the Catalog page list these services directly. Software engines such as PostgreSQL or Redis are not palette items: they are a configuration of a service (for example the engine of RDS). The AWS and GCP lists confirm candidate names against the official product pages; the Azure list is the full official list. Every service name links to the official page used to verify it, read Oct 20, 2026. A link shows where to look; it does not mean the limits were read. Dataset licenses are not checked.

Status labels

- **Verified**: limits read and recorded in the Verified-limits table (pp. 21–23) or the Oct 18, 2026 AWS spot check (Lambda, SQS, S3, CloudFront).
- **Partial**: the page states some limits but not all, or the figures are guidance rather than quotas.
- **Listed (not checked)**: an official limits page exists and is linked, but its figures are not yet recorded in the plan.
- **No limits on page**: the linked official page states no limits (concept or overview pages).

Changes on Oct 20, 2026

- 17 services were marked "limits checked" without a recorded reading. 13 of them are now Listed (not checked): AWS DynamoDB, API Gateway, SNS, EventBridge; Azure Functions, Blob Storage, Cosmos DB, Event Grid, API Management; GCP Cloud Run, Cloud Run functions, Cloud Storage, Firestore. The other 4 (Azure Front Door, Azure Content Delivery Network, Cloud CDN, Apigee) take the status their pages support, in the next point.
- 6 services changed because of what their pages state (the 4 above plus S3 and Azure Load Balancer): S3, Azure Front Door, Azure Load Balancer, Cloud CDN and Apigee are now Partial; Azure Content Delivery Network is now No limits on page.
- GCP Pub/Sub: the old link returned 404. It is replaced by the confirmed official page https://docs.cloud.google.com/pubsub/quotas ("Pub/Sub quotas and limits").
- GCP Dataproc: Google renamed it "Managed Service for Apache Spark"; the old link redirects to https://docs.cloud.google.com/managed-spark/quotas. The catalog keeps the name Dataproc and notes the new name.
- Naming: use "Application Load Balancer" everywhere; "ALB" is allowed only as a short label. The canvas sources "Viewers" and "Uploaders" are labels on the Generic **Clients** component, not catalog services. ECR is not a catalog entry.
- Kubernetes: the 7 objects are kinds with no limits on their pages. Cluster size limits get their own row ("Kubernetes scalability thresholds", Partial).

Counts

Provider	Services	Verified	Partial	Listed (not checked)	No limits on page
AWS	50	9	1	40	0
Azure	44	4	2	36	2
Google Cloud	35	5	2	28	0
Kubernetes	8	0	1	0	7
Total	**137**	**18**	**6**	**104**	**9**

AWS (50 services)

Category	Service (links to its official page)	Limits on page	Status	Note
Compute	EC2	yes	Listed (not checked)	
	Lambda	yes	Verified	
	Fargate	yes	Listed (not checked)	Indirect: ECS throttling page (Fargate launch rates).
	Batch	yes	Listed (not checked)	
	App Runner	yes	Listed (not checked)	
	Elastic Beanstalk	yes	Listed (not checked)	
	Lightsail	yes	Listed (not checked)	
Containers	ECS	yes	Verified	
	EKS	yes	Listed (not checked)	
Storage	S3	partial	Partial	Performance guidance, not quotas.
	EBS	yes	Listed (not checked)	
	EFS	yes	Listed (not checked)	

	FSx	yes	Listed (not checked)	Indirect: FSx for Windows only.
	Storage Gateway	yes	Listed (not checked)	
Database	Aurora	yes	Verified	
	RDS	yes	Verified	
	DynamoDB	yes	Listed (not checked)	
	DocumentDB	yes	Listed (not checked)	
	Neptune	yes	Listed (not checked)	
	Keyspaces	yes	Listed (not checked)	
	Timestream	yes	Listed (not checked)	
	Redshift	yes	Listed (not checked)	
Cache	ElastiCache	yes	Verified	
	MemoryDB	yes	Verified	
Network and CDN	CloudFront	yes	Verified	
	Route 53	yes	Listed (not checked)	
	Application Load Balancer	yes	Verified	
	Network Load Balancer	yes	Listed (not checked)	
	API Gateway	yes	Listed (not checked)	
	Global Accelerator	yes	Listed (not checked)	
	VPC	yes	Listed (not checked)	
	Direct Connect	yes	Listed (not checked)	

	WAF	yes	Listed (not checked)	
	Shield	yes	Listed (not checked)	
Messaging and integration	SQS	yes	Verified	
	SNS	yes	Listed (not checked)	
	EventBridge	yes	Listed (not checked)	
	Kinesis Data Streams	yes	Listed (not checked)	
	MSK	yes	Listed (not checked)	
	Step Functions	yes	Listed (not checked)	
	MQ	yes	Listed (not checked)	
	AppSync	yes	Listed (not checked)	
Analytics	OpenSearch Service	yes	Listed (not checked)	
	Athena	yes	Listed (not checked)	
	EMR	partial	Listed (not checked)	
	Glue	yes	Listed (not checked)	
Media	MediaConvert	yes	Listed (not checked)	
	MediaLive	yes	Listed (not checked)	
	MediaPackage	yes	Listed (not checked)	
	IVS	yes	Listed (not checked)	

Azure (44 services)

Category	Service (links to its official page)	Limits on page	Status	Note
Compute	Virtual Machines	partial	Listed (not checked)	
	VM Scale Sets	yes	Listed (not checked)	Indirect: FAQ page.
	App Service	yes	Listed (not checked)	Generic Azure subscription limits page.
	Functions	yes	Listed (not checked)	
	Batch	yes	Listed (not checked)	
	Container Instances	yes	Listed (not checked)	
Containers	AKS	yes	Verified	
	Container Apps	partial	Listed (not checked)	
Storage	Blob Storage	yes	Listed (not checked)	
	Files	yes	Listed (not checked)	
	Disk Storage	yes	Listed (not checked)	
	NetApp Files	yes	Listed (not checked)	
	Data Lake Storage	no	No limits on page	Product page, no limits.
	Elastic SAN	yes	Listed (not checked)	
Database	Cosmos DB	yes	Listed (not checked)	
	Azure SQL	yes	Verified	vCore single-database limits only.
	SQL Managed Instance	yes	Listed (not checked)	

	Database for PostgreSQL	yes	Listed (not checked)	
	Database for MySQL	partial	Listed (not checked)	
	Managed Instance for Apache Cassandra	partial	Listed (not checked)	
Cache	Cache for Redis	yes	Verified	Generic Azure subscription limits page. Basic, Standard, Premium retire Sep 30, 2028.
	Managed Redis	partial	Listed (not checked)	
Network and CDN	Front Door	partial	Partial	
	Content Delivery Network	no	No limits on page	Overview page, no limits.
	Application Gateway	yes	Listed (not checked)	
	Load Balancer	partial	Partial	Indirect: idle-timeout how-to.
	Traffic Manager	partial	Listed (not checked)	Indirect: FAQ page.
	DNS	yes	Listed (not checked)	
	ExpressRoute	yes	Listed (not checked)	
	Firewall	yes	Listed (not checked)	
	Web Application Firewall	partial	Listed (not checked)	
	VPN Gateway	yes	Listed (not checked)	
Messaging and integration	Service Bus	yes	Verified	
	Event Grid	yes	Listed (not checked)	
	Event Hubs	yes	Listed (not checked)	

	Queue Storage	yes	Listed (not checked)	
	API Management	yes	Listed (not checked)	
	Logic Apps	yes	Listed (not checked)	
	Web PubSub	partial	Listed (not checked)	
Analytics	Data Explorer	yes	Listed (not checked)	Generic Azure subscription limits page.
	Stream Analytics	partial	Listed (not checked)	
	Synapse Analytics	yes	Listed (not checked)	
	Databricks	partial	Listed (not checked)	
	Data Factory	yes	Listed (not checked)	Generic Azure subscription limits page.

Google Cloud (35 services)

Category	Service (links to its official page)	Limits on page	Status	Note
Compute	Compute Engine	partial	Listed (not checked)	
	Cloud Run	yes	Listed (not checked)	
	Cloud Run functions	yes	Listed (not checked)	
	App Engine	yes	Listed (not checked)	
	Batch	yes	Listed (not checked)	
Containers	GKE	yes	Verified	
Storage	Cloud Storage	yes	Listed (not checked)	
	Persistent Disk	yes	Listed (not checked)	

	Filestore	yes	Listed (not checked)	
	Hyperdisk	yes	Listed (not checked)	
Database	Cloud SQL	yes	Verified	
	AlloyDB	yes	Listed (not checked)	
	Spanner	yes	Listed (not checked)	
	Bigtable	yes	Listed (not checked)	
	Firestore	yes	Listed (not checked)	
Cache	Memorystore	yes	Verified	
Network and CDN	Cloud CDN	partial	Partial	
	Media CDN	yes	Listed (not checked)	
	Cloud Load Balancing	yes	Verified	
	Cloud DNS	yes	Listed (not checked)	
	Cloud Armor	yes	Listed (not checked)	
	Cloud NAT	yes	Listed (not checked)	
	Cloud Interconnect	yes	Listed (not checked)	
Messaging and integration	Pub/Sub	yes	Verified	Link fixed: the old URL returned 404; new page confirmed Oct 20, 2026.
	Cloud Tasks	yes	Listed (not checked)	
	Eventarc	yes	Listed (not checked)	
	Workflows	yes	Listed (not checked)	

	Cloud Scheduler	yes	Listed (not checked)	
	Apigee	partial	Partial	
	API Gateway	yes	Listed (not checked)	
Analytics	BigQuery	yes	Listed (not checked)	
	Dataflow	yes	Listed (not checked)	
	Dataproc	yes	Listed (not checked)	Renamed by Google: "Managed Service for Apache Spark"; link updated.
Media	Transcoder API	yes	Listed (not checked)	
	Live Stream API	yes	Listed (not checked)	

Kubernetes (8 services)

Category	Service (links to its official page)	Limits on page	Status	Note
Kubernetes object	Deployment	no	No limits on page	
	StatefulSet	no	No limits on page	
	Job	no	No limits on page	
	Service	no	No limits on page	
	Ingress	no	No limits on page	
	HorizontalPodAutoscaler	no	No limits on page	
	PersistentVolumeClaim	no	No limits on page	
Cluster limits	Kubernetes scalability thresholds	partial	Partial	

Generic (9 components)

Group	Components
Generic	Clients, Load balancer, Service, Cache, Database, Queue, Worker pool, CDN, Object store (no provider page: generic building blocks; limits Not set until entered). The canvas labels "Viewers" and "Uploaders" are Clients.

Catalog value rules

These rules decide how a catalog value is stored. They were set after checking AWS documentation: 11 figures (Lambda, SQS, S3, CloudFront) were checked on Oct 18, 2026 and all 11 match. The other claims in the source answer are not checked. The workbook aws_catalog_verification.xlsx holds the full list, and your Azure and GCP checks go in its last sheet.

What changes

#	Change	Replaces	Status of the evidence
M1	Every catalog value carries a limit type: hard limit, adjustable quota, best-effort target or performance guidance	Confidence label alone	Spot-checked: S3 rates are guidance, Lambda concurrency is adjustable

#	Change	Replaces	Status of the evidence
M2	Every catalog value also stores unit, scope, conditions, source URL and retrieval date	Value plus confidence plus source id	Proposed by ChatGPT; adopted here
M3	API Gateway splits into REST, HTTP and WebSocket entries	One entry	ChatGPT; not checked by me
M4	SQS splits into Standard and FIFO	One entry	Size and retention checked; delivery semantics not
M5	EC2, ECS and EKS each need their own configuration (launch model, resources, scaling policy)	Treated as one kind of service	ChatGPT; not checked by me
M6	Lambda limits are stored per execution model; 900 s applies to ordinary functions only	One duration limit	Checked: Managed Instances allow 5,400 s for asynchronous and event-source-mapping invocations
M7	MemoryDB is a durable database profile; ElastiCache is not labelled non-durable in general	One cache entry	ChatGPT; not checked by me
M8	DynamoDB is modeled by capacity mode, item size, indexes and access pattern, not one partition limit	Partition limit	ChatGPT; not checked by me
M9	RDS and Aurora get no queries-per-second value without engine, instance class, workload and a benchmark source	None	ChatGPT; not checked by me
M10	Scaling delay has no default; it is configurable or measured	A delay field	ChatGPT; not checked by me

Catalog value type

This compiled under tsc --strict, and the worked check returned the expected values: a Lambda timeout of 900 for a standard function, 5,400 for Managed Instances asynchronous invocation, and S3 request rates refused as a hard ceiling.

```
type LimitType = 'hard_limit' | 'adjustable_quota' | 'best_effort_target' |
'performance_guidance';
type Status = 'documented' | 'measured' | 'estimated' | 'unknown';

interface CatalogValue {
  value: number;
  unit: string;
  scope: 'function' | 'account' | 'region' | 'distribution' | 'queue' |
'message' | 'prefix' | 'invocation';
  limitType: LimitType;
  conditions: Record<string, string>; // API type, engine, instance family,
execution model
  sourceUrl: string;
  retrievedOn: string; // YYYY-MM-DD
  status: Status;
}

// Rule: only hard limits and adjustable quotas may act as a ceiling in the
engine.
function usableAsCeiling(v: CatalogValue): boolean {
  return v.limitType === 'hard_limit' || v.limitType === 'adjustable_quota';
}
```

Rule proposed, not confirmed: performance guidance and best-effort targets are shown in the UI and used as defaults a user can override, but they are never enforced as a hard cap. The confidence labels in Calibration map onto status: Measured to measured, Vendor figure to documented, Estimated to estimated.

Provider-mapping rows, with the configuration each needs

Kind	AWS entries	Configuration required
Gateway	API Gateway REST, HTTP, WebSocket	API type, throttling, payload, timeout
Load balancer	ALB, NLB	Layer and protocol, targets, connections

Kind	AWS entries	Configuration required
Service	EC2; ECS by launch model; EKS by compute and pod settings	Capacity, orchestration, scaling policy, warmup
Function	Lambda	Execution model, invocation type, concurrency, initialization
Queue	SQS Standard, SQS FIFO	Delivery semantics, ordering, message size, retention
Event bus	EventBridge, SNS	Routing versus topic fan-out, retries, dead-letter handling
Cache	ElastiCache; MemoryDB as a database profile	Engine, node type, eviction, replication, durability
Relational database	RDS by engine; Aurora by compatibility	Instance class, connections, replicas, failover
NoSQL database	DynamoDB	Capacity mode, item size, indexes, access pattern, consistency
Object storage	S3	Request mix, prefix distribution, object size
CDN	CloudFront	Cache behavior, hit ratio, origin behavior, quotas

Decided Oct 19, 2026: the Azure and GCP names were checked against official pages (linked under Components by provider); Pub/Sub is one entry listed under both Queue and Event bus.

Verified limits

The 18 services of the first reference architecture (load balancer, service, queue, cache, database) were read on official AWS, Azure and GCP pages; the table below lists what each page stated. Unknown means the page read did not state a value, not that no limit exists. These are the 18 Verified services in Components by provider (Oct 20, 2026 audit of 137 services: 18 Verified, 6 Partial, 104 Listed (not checked), 9 No limits on page). Sources are in verify_checklist.xlsx; re-check any figure before it ships.

Service	Verified	Still open
AWS ALB	Idle timeout 60 s default, range 1 to 4,000 s; 1,000 targets, 50 listeners, 100 rules (adjustable)	Scaling time; connection rate
AWS ECS	5,000 tasks per service, 5,000 services per cluster (fixed); 10 tasks per RunTask; Fargate launch 20 per second sustained (adjustable)	Cooldown defaults; warm-up
AWS SQS Standard	At-least-once delivery, best-effort order, nearly unlimited API calls	Numeric quota
AWS SQS FIFO	Strict order within a message group; 5-minute deduplication; 3,000 messages per second per partition batched, 300 unbatched (guidance); 120,000 in flight	Regional total
AWS ElastiCache	Asynchronous replication, writes resume in seconds after failover; serverless caches 65,000 clients	Node-based connections and eviction
AWS MemoryDB	Multi-AZ transaction log; no data lost on failover	Node sizes, connections, eviction
AWS RDS	Multi-AZ instance is failover only; Multi-AZ cluster also serves reads; 15 read replicas per primary	Per-instance storage; consistency
AWS Aurora	Up to 15 replicas; failover usually under 60 s; MySQL connection defaults by class	Storage (128 or 256 TiB, pages differ)
Azure Load Balancer	Idle timeout 4 min default, 4 to 100 (outbound 4 to 120)	Limits; scaling

Service	Verified	Still open
Azure AKS	5,000 nodes, 1,000 per pool, 100 pools, 250 pods per node; autoscaler scan 10 s	Scale-up time
Azure Service Bus	Peek-lock at-least-once; sessions give ordering	Throughput quota
Azure Cache for Redis	Connections 256 to 40,000 by tier; volatile-lru; asynchronous replication	Retires Sep 30, 2028 (Basic, Standard, Premium) and Mar 31, 2027 (Enterprise)
Azure SQL	30,000 sessions; 150 and 600 workers (serverless General Purpose); Business Critical up to 3 secondaries	Failover time; other tiers
GCP Cloud Load Balancing	Backend timeout 30 s default; 50 backends per backend service	Connection rate; scaling
GCP GKE	Standard 65,000 nodes, Autopilot 5,000; 512 pods per node	Autoscaler timing
GCP Pub/Sub	At-least-once, optional ordering, exactly-once supported; 4 GB/s per region publish in large regions	None found
GCP Memorystore	65,000 clients; asynchronous, about 30 s unavailable on repair; acknowledged writes can be lost	Memory sizes
GCP Cloud SQL	Synchronous HA, about 60 s failover; PostgreSQL connection sum up to 262,142; storage up to 64 TB	Standby reads; consistency

Rules this adds to the catalog (proposed)

- Status gains "Partial" (some of the item found). Limit type gains "configurable setting", "documented behavior" and "quota, adjustability not checked".

- A value can depend on the deployment: ElastiCache serverless versus node-based, Aurora engine version, Azure SQL tier. Store that as a condition, not a footnote.
- Pub/Sub is one catalog entry offered under both Queue and Event bus. Not yet confirmed by you.
- A variant carries a retirement status and date (for example Azure Cache for Redis).
- Figures that only ChatGPT reported, such as a Google load balancer timeout maximum of 86,400 s, stay unverified until a page confirms them.

Simulation engine

The engine runs one topology in two modes: a fast fluid-flow pass for feasibility and cost, and a discrete-event pass for tail latency and failure cascades.

	Fluid-flow	Discrete-event
Data unit	Rate vector per request class	Individual request objects
Cost	Very cheap, scales to millions of users	Higher, grows with request count
Answers	Capacity and throughput curves, rough cost	P99 latency, queue saturation, retries, cascades
Use it for	First feasibility check and sizing	Bottlenecks, bursts, fault injection

Fluid-flow runtime model

Fluid-flow mode tracks rates per request class, in small time steps Δt (for example 100 ms). It answers "does this design have room?" quickly, at high traffic, without simulating single requests.

Runtime state. For each edge and class the engine keeps five rates (`FluidEdgeState`): offered, delivered (after network loss and bandwidth), admitted (accepted by the receiver), completed, and retried. For each component and class it keeps a backlog B. Dropped is not stored; it is the difference, so the counts cannot disagree.

One step, in order

1. **Offered.** Compute each edge's offered rate from its source with the routing formulas in the shared edge contract, plus the retry rate from the previous step.
2. **Network.** Delivered = offered × (1 − lossProb), then capped by bandwidth divided by bytes per request. The difference is a network drop.
3. **Admission.** The receiver has capacity μ = c / s̄ per second, where c is `servers` and s̄ the mean service time (mixed over the classes). It admits what fits in service plus the free queue room; the rest is an application rejection.
4. **Service.** Completed = min(admitted + B/Δt, μ). The backlog updates: B(t+Δt) = B(t) + (admitted − completed) Δt.
5. **Cache.** A cache removes the hit fraction h from the downstream flow: downstream = (1 − h) × admitted.
6. **Retries.** Failed = network drops + rejections + timeouts. Retried next step = min(failed × A-factor share, budget × first attempts), if the class is eligible (idempotent or allowed).

Because retries and loops can feed back, the engine iterates steps 1 to 4 to a fixed point inside each Δt, with a small damping factor, and stops after a set number of passes (a decision to confirm).

Utilization and waiting. ρ = λ / μ. For the report, the expected wait comes from the M/M/c result (Erlang C) as an approximation, labeled as such, because fluid mode has no individual requests.

Worked example: an overloaded worker pool (illustrative). 16 slots, 100 ms per event, so μ = 160 per second. Waiting room K = 50.

Offered	Utilization	What happens
150 per second	94%	Stable; queue stays short
200 per second	125%	Backlog grows by 40 per second and fills the 50 places in 1.25 s; after that 40 per second are rejected

Conservation, checked every step, for each component and class:

```
\text{offered} = \text{admitted} + \text{rejected} + \text{network drop},\qquad \Delta B = (\text{admitted} - \text{completed})\,\Delta t
```

Backpressure is this admission step, not a hidden multiplier: a full queue rejects, the rejection is counted, and the caller's retry and circuit-breaker settings decide what happens next.

Discrete-event runtime model

Discrete-event mode follows individual requests. It answers "what do queues, bursts, retries, timeouts and failures do to the tail latency?", which averages cannot show. It reads the same topology and the same component and edge definitions as fluid mode.

A request carries: a request id and a trace id, its class, request and response bytes, creation time, deadline and priority (when set), attempt count, and the path taken so far. Retries keep the trace id and increase the attempt count.

The scheduler is a priority queue of events ordered by timestamp, with a tie-break by a sequence number so replays are exact. There is no blocking loop: the engine pops the next event, handles it, and may schedule new ones. A random generator seeded from the run seed supplies all randomness, one stream per component, so adding a component does not change another's draws.

Event	What it does
`arrive`	A request reaches a component. If a server is free it starts service; else it joins the queue; if the queue is full it is rejected (application rejection)
`start-service`	Draws a service time from the class distribution and schedules `finish-service`
`finish-service`	Frees the server, takes the next queued request (by priority, then arrival), and routes the finished one along its edges
`send`	Applies network latency, loss (a network drop), serialization if on, and bandwidth queueing; schedules `arrive` at the target
`timeout`	Fires when a sync call has waited too long. The request is marked timed out; a retry is scheduled if eligible
`retry`	Schedules a new attempt after the backoff, within the retry budget and the deadline

Event	What it does
`breaker-change`	Moves a circuit breaker between closed, open and half-open
`fault`	Starts or ends an injected fault: crash, latency spike, link cut, loss

Deadlines and priority. A request past its deadline is dropped at the next event and counted as expired. Within a queue, higher priority goes first.

Cache. A cache holds keys. Each cacheable request draws a key from the configured popularity distribution; a hit is served by the cache and a miss continues downstream and fills the cache. The configured aggregate hit rate in fluid mode is the target the key model must reproduce, within a stated tolerance, in a steady state.

Metrics. Every request logs its path and timestamps, so P50, P90 and P99, queue depth over time, and per-class counts all come from the same record, and a single trace can be opened and replayed.

Where the modes differ, on purpose. Under bursts, priorities, deadlines, finite queues and random variation, discrete-event mode will show higher tail latency and earlier rejections than fluid mode. That is the reason both exist. They are compared only in stable, light-load cases (N7).

Calibration

A number is trusted in proportion to its evidence, and the screen always shows which evidence it has. The simulator ships no invented benchmark: a value either has a source, or it is an estimate the user typed, labeled Estimated.

Confidence labels, in order of trust

Label	Meaning	Example source
Measured	Taken from a load test of this service, with the setup recorded	A user's imported CSV; a published test with its raw data
Vendor figure	Published by the provider or the software's maintainers	A documented quota or a vendor benchmark page

Label	Meaning	Example source
Estimated	Entered or defaulted, with no external evidence	A user's best guess; a placeholder to replace

When several profiles apply to one component, the simulator uses the most trusted one and shows the others as alternatives. It never blends them silently.

A profile records: the component kind, source URL or dataset name, capture date, software or service version, region and tier or SKU, request class, the configuration under test, the values (service-time distribution, saturation point, limits), the label, and who added it.

Importing a load test. The user uploads a CSV of requests or of intervals. A proposed minimum is below; the exact format is a decision to confirm.

Column	Needed for
`timestamp`	Ordering and arrival rate
`requestClass`	Service time per class
`latencyMs`	Service-time distribution
`status`	Error and rejection rates
`concurrency` or `rps`	Where latency starts to climb: the saturation point

From the file the app fits a service-time distribution per class, finds the throughput at which latency rises sharply, and shows the fit next to the data so the user can judge it. A fit with few samples is labeled low-sample.

Typing values by hand (F28, decided Oct 19, 2026). A user does not need a load test. Each profile has editable fields for the median and P95 service time per request class, the connection pool and other capacity inputs, and can add custom request classes. The label follows the evidence: no source gives Estimated, a source link gives Vendor figure, an imported CSV gives Measured. Typed values never get the Measured label. The writes-per-second bound is calculated from the typed values, not typed. The CSV hint reads "concurrency or requests per second", matching the table above. Decided Oct 19, 2026: any valid link is accepted and its date recorded; the label shows Vendor figure with "source not checked" until confirmed.

Checking the simulator against real data. Fidelity (N1) is measured by simulating a recorded test's workload and comparing which components saturate first. The correlation metric and the dataset are open decisions (see Decisions). The Blueprint's 92% target is dropped for now, and the app claims no fidelity number until the metric and dataset are set.

Freshness. Profiles show their age. Provider quotas and prices carry a read date and are flagged after a set time (to be decided).

Separation rule. The service catalog says what a service can be configured to do. The calibration library says what it measured. A catalog entry is never used as a measurement.

Validation and tests

Every test below is written before the code it checks. Where a tolerance cannot be known before a first measurement, the Status column says so; it is fixed once and recorded before any comparison run, because a tolerance picked after seeing results proves nothing.

#	Test	Setup	Pass when	Status
J1	Flow conservation, fluid	Any topology, every step, every component and class	offered = admitted + rejected + network drop, and $\Delta B = (\text{admitted} - \text{completed})\,\Delta t$, to floating-point precision	Defined
J2	Flow conservation, discrete-event	Any run	Requests created = completed + rejected + dropped + expired + still in flight at the end	Defined
J3	Deterministic replay	Same topology version, workload, mode, seed and calibration, run twice	The two event logs are byte-identical	Defined

#	Test	Setup	Pass when	Status
J4	Queueing theory	One component, Poisson arrivals, exponential service times, c servers	Fluid utilization equals λ/(cμ); discrete-event mean wait matches the M/M/c (Erlang C) value within the tolerance over many seeds	Decided: very tight tolerance (exact formula)
J5	Mode agreement	Stable, light load (utilization below about 50%) on the reference architecture	Utilization and throughput agree within the tolerance	Decided: 5% to start (a labeled guess), tightened later
J6	Routing examples	The weighted, broadcast and fan-out cases of the shared edge contract	The offered rates match the worked numbers exactly	Defined
J7	Retry behavior	Failing dependency with f = 0.1 and 2 retries	Attempts per request converge to 1.11; retries never exceed the budget	Budget value fixed after the first measurement
J8	Cache hit rate	Discrete-event cache with a key distribution	Measured hit rate reproduces the configured aggregate within the tolerance	Tolerance fixed after the first measurement
J9	Fault: node crash	Crash a component mid-run	Its inflow becomes rejections or timeouts from that moment; blast radius lists the affected components	Defined

#	Test	Setup	Pass when	Status
J10	Fault: link cut, loss, latency spike	Cut a link; set loss p; add a delay	A cut gives 100% network drop on that edge; measured loss approaches p; latency rises by the added delay	Defined
J11	Circuit breaker	Failing dependency	Transitions follow the rule below	Decided Oct 19, 2026
J12	Fidelity	Recorded load test of the reference architecture	Simulated and measured bottleneck ranking correlate above the KPI	Decided: A and B pass, C diagnostic; B tolerance after the first measurement

Circuit-breaker rule (decided Oct 19, 2026). Closed to open when the error rate over a sliding window reaches the threshold and at least a minimum number of requests were seen. Open to half-open after the open duration. In half-open, a small number of probe requests are allowed: if they succeed the breaker closes, and if one fails it opens again. Requests rejected while open are counted as breaker rejections, separate from application rejections. All parameters are configurable with documented defaults.

An example in J4, from the earlier worked numbers: 16 servers, 100 ms service time and 150 arrivals per second give utilization 93.75% and an expected wait of about 73 ms. The test recomputes this value; it is not assumed.

Architecture and stack

The simulation runs in the browser and the backend only stores data, so v1 needs four layers. Where the two source docs disagree, this table picks one and says why.

Layer	Choice	Why	Status
UI	React 18, TypeScript, React Flow canvas	Both docs agree	Source

Layer	Choice	Why	Status
UI state	Zustand for visual state; engine state kept inside the worker	Prevents re-render storms from simulation ticks	Source: blueprint
Engine language	TypeScript in a Web Worker first, behind a stable engine interface; port the hot loop to Rust/WASM only if profiling shows it is needed	Docs disagree (Rust/WASM versus TypeScript). TypeScript is faster to build and validate; the interface keeps the later swap cheap	Proposed
UI to engine messaging	Batched snapshots by message passing; move to SharedArrayBuffer only if measured necessary	SharedArrayBuffer needs cross-origin isolation headers on hosting	Proposed
Charts	Recharts or Chart.js; choose one in Phase 2	Blueprint lists both	Open
Backend	Node.js API: auth, topology save and load, calibration library, CSV import	Matches blueprint; no simulation runs here	Source
Storage	PostgreSQL; add Redis only when sessions or caching need it	Blueprint lists Redis for sessions; not required for a first version	Proposed

The blueprint's WebSocket or gRPC-Web link between the engine and backend has no purpose while the simulation is client-side, so it is dropped from v1 and kept as an option for server-side runs of very large models.

Engine interfaces

The types below follow the System model and Simulation engine sections. They compile under `tsc --strict`, and the checks at the end reproduce the worked numbers of the shared edge contract (all six returned true when run). Values that wait on a first measurement are marked in a comment. Nothing in the core names a cloud provider; the provider appears only in the optional binding.

Component and value types

```
export type RequestClassId = string; // defaults: read, write, async, bulk;
custom allowed
export type Confidence = "measured" | "vendor" | "estimated";

/** Set (with evidence), unknown, or not applicable. Never silently
defaulted. */
export type Value<T> =
  | { state: "set"; value: T; confidence: Confidence; sourceId?: string }
  | { state: "unknown" }
  | { state: "not_applicable" };

export type Distribution =
  | { kind: "constant"; ms: number }
  | { kind: "exponential"; meanMs: number }
  | { kind: "lognormal"; medianMs: number; sigma: number };

export type CanonicalKind =
  | "traffic_generator" | "gateway" | "load_balancer" | "service" | "worker"
  | "queue" | "event_bus" | "cache" | "relational_db" | "nosql_db"
  | "object_storage" | "cdn";

export interface Component {
  id: string;
  name: string;
  kind: CanonicalKind;
  behavior: {
    servers: Value<number>;
    serviceTime: Record<RequestClassId, Value<Distribution>>;
    queueCapacity: Value<number | "unbounded">;
    throughputLimitRps: Value<number>;
    scaling: Value<{ min: number; max: number; delaySec: number } | "fixed">;
    cacheHitRate?: Value<number>; // caches only
  };
  providerBinding?: ProviderBinding;
  calibrationProfileIds: string[];
}

export interface ProviderBinding {
  provider: "aws" | "azure" | "gcp" | "kubernetes" | "self_hosted";
  service: string; // catalog entry id
  sku?: string;
```

```
  region?: string;
  quotas: { name: string; value: number; sourceUrl: string; readOn: string }
[];
}
```

Shared edge contract

```
export type Routing =
  | { mode: "single" }
  | { mode: "weighted"; weight: number }
  | { mode: "broadcast" }
  | { mode: "conditional"; classes: RequestClassId[] }; // v1: by class only

export interface Edge {
  id: string;
  from: string;
  to: string;
  routing: Routing;
  fanOut: number; // calls per routed request, default 1; not a probability
  classes: Record<RequestClassId, { requestBytes: number; responseBytes:
number }>;
  network: {
    latencyMs: number;
    bandwidthBps?: number;
    bandwidthApplies: "request" | "response" | "both";
    lossProb: number; // network drop only
    serialization: boolean;
  };
  call: { kind: "sync" | "async"; timeoutMs?: number; deadlineMs?: number };
  retry?: { max: number; backoff: "fixed" | "exponential"; baseMs: number;
            budget: number /* fixed after first measurement */;
onlyIdempotent: boolean };
  breaker?: { errorThreshold: number; windowMs: number; openMs: number;
probes: number }; // rule decided Oct 19, 2026
}
```

Runtime state, both modes

```
/** Fluid mode: rates per second, one record per edge and class. */
export interface FluidEdgeState {
  offered: number;
  delivered: number; // after network loss and bandwidth
  admitted: number;  // accepted by the receiver
```

```
  completed: number;
  retried: number;
} // dropped is derived (offered minus delivered is network drop; delivered
minus admitted is rejection), never stored

/** Discrete-event mode. */
export interface SimRequest {
  id: string; traceId: string; classId: RequestClassId;
  requestBytes: number; responseBytes: number;
  createdAtMs: number; deadlineMs?: number; priority?: number;
  attempt: number; path: string[];
}

export type SimEvent =
  | { t: number; seq: number; type: "arrive"; req: SimRequest; at: string }
  | { t: number; seq: number; type: "finish-service"; req: SimRequest; at:
string }
  | { t: number; seq: number; type: "timeout"; req: SimRequest; edge: string
}
  | { t: number; seq: number; type: "retry"; req: SimRequest; edge: string }
  | { t: number; seq: number; type: "breaker-change"; edge: string; to:
"closed" | "open" | "half_open" };
```

Worked examples as checks

```
export function offeredRate(
  routing: Routing, fanOut: number, admitted: number, group: number[] = [],
): number {
  switch (routing.mode) {
    case "single":
    case "broadcast": return fanOut * admitted;
    case "weighted": {
      const sum = (group.length ? group : [routing.weight]).reduce((a, b) =>
a + b, 0);
      return (routing.weight / sum) * fanOut * admitted;
    }
    case "conditional": return fanOut * admitted; // admitted is already the
class share
  }
}

/** Attempts per request: sum of f^i for i = 0..r. */
export const attemptsPerRequest = (f: number, r: number): number =>
```

```
  f === 1 ? r + 1 : (1 - Math.pow(f, r + 1)) / (1 - f);

export function conservationHolds(
  s: { offered: number; admitted: number; rejected: number; networkDrop:
number },
  eps = 1e-9,
): boolean {
  return Math.abs(s.offered - (s.admitted + s.rejected + s.networkDrop)) <=
eps;
}

// Expected: weighted 200 and 600; broadcast 150; fan-out 400; retries 1.11;
conservation true
```

Before building: `FluidEdgeState`'s drop counts are derived from the stored rates, and the tests for J4, J5 and J8 need their tolerances (see Validation and tests) before they can be written.

Data model

Seven entities cover v1: a topology owns components and edges, a workload and a run use them, and calibration profiles feed the components.

Entity	Holds	Links to
Topology	name, owner, version, canvas layout	Has many Components and Edges
Component	type, position, concurrency, capacity, per-class service times, failure settings	Belongs to a Topology; references a Calibration profile
Edge	source, target, fan-out multiplier, routing probability, latency, bandwidth, per-class rate, payload size, call semantics	Connects two Components
Request class	name, default cost and cacheability	Used by Edges, Workloads and Profiles

Entity	Holds	Links to
Workload	load shape (constant, Poisson, step, spike), parameters, per-class mix, seed	Applies to a Topology
Calibration profile	component type, request class, service-time distribution, pool size, max throughput, hardware, version, date, source, confidence	Referenced by Components
Run	topology version, workload, mode, seed, metrics time series, invariant results	Belongs to a Topology; reproducible from its seed

A Run stores the topology version and seed it used, so any past result can be replayed exactly.

Roadmap

Build the engine and prove it against theory and one real stack first, then add the canvas around it. Each phase ends only when its exit criteria pass. Durations are left open until the team size is known.

Phase	Scope	Exit criteria
0. Lock requirements	Apply the decisions in Decisions; fix request classes and limits (N6); sign off the edge contract and the circuit-breaker rule	Decisions recorded; requirements table signed off
1. Engine core and validation	Graph and matrix structures, canonical component schema (F15), generic catalog (F16), edge contract (F19), discrete-event kernel, typed requests, invariants, one calibrated reference stack built from generic components; tests J1 to J3, J6, J7, J9 to J11	Conservation of flow and deterministic replay pass; M/M/c results match theory; reference stack predicted against a real load test

Phase	Scope	Exit criteria
2. Canvas, config and telemetry	React Flow canvas, provider services in the palette with mixed-provider designs, config panel (unknown and estimated fields shown), workload editor, request-class filter (F24), worker integration, live metrics and heatmap	A user builds the reference stack and runs it; UI holds 60 FPS during a run
3. Fluid-flow mode and calibration	Fluid-flow mode, calibration library with CSV import and typed entry, bottleneck explainer (F23), tests J4, J5, J8, first provider catalog as data (F17)	Modes agree on a stable scenario within the set tolerance; every number in a run shows its profile source
4. Faults and persistence	Fault injection, blast-radius view, save and load, auth, template library and tutorials (F22, F20), cross-provider comparison (F18, capacity and limits), cost estimate (F11), more catalogs	Chaos checks trip upstream breakers; topologies survive reload and replay exactly
5. Fidelity validation and hardening	Compare reference architectures with real load tests, including the reference architecture mapped to each provider; enforce limits; test J12	Bottleneck predictions checked against real load tests; gaps documented per profile
Later	Nested sub-systems, IaC export (Terraform, CloudFormation), multi-region networking, server-side runs, challenge mode (F21), topology import (F25)	Scheduled after Phase 5 results

Risks

The biggest risk is a simulator that looks precise but is wrong; the calibration layer and early validation exist to address it.

Risk	Effect	Mitigation
Calibration data is thin or stale	Confident but wrong predictions	Show source and confidence on every number; import the user's own load tests; early validation loop
Fidelity goal is hard to define or measure	KPI cannot be proven	Define the correlation measure in Phase 0; report fidelity per profile
Discrete-event runs are too slow at large scale	Poor interactivity	Use fluid-flow mode for scale; limit request counts; profile before moving to Rust/WASM
Models too simple (single-queue approximations) miss real behavior	Missed bottlenecks	Typed requests, retries and backpressure modeled from Phase 1
Scope grows toward a general infrastructure tool	Build never finishes	Keep IaC export and nested systems out of v1

Decisions

Every open design and architecture item in this plan, grouped for discussion. All 17 items were decided on Oct 19, 2026 by accepting my recommendation; each shows its decision under the item. Who the users are is out of scope for now (Oct 19, 2026): decisions here follow from the design and architecture alone.

Each item has four lines: what it is, the options, my recommendation, and what it blocks. To change one, tell me in chat and I will update the item and what it blocks.

Group 1. Design and architecture choices

1. Which provider ships first

- **What it is.** The engine and generic catalog are cloud-agnostic (F16). The first release still needs one provider catalog filled in with sourced data (F17).
- **Options.** AWS, Azure, GCP or Kubernetes. Priority 1 checking so far: AWS has 26 rows (10 verified, 9 partial, 7 unknown), Azure 16 (7, 6, 3), GCP 16 (10, 2, 4). Kubernetes has not been checked.

- **Recommendation.** AWS first. It has the most rows checked, and the Phase 1 catalog entries (Lambda, SQS, S3, CloudFront) are AWS. The cost is its 7 unknown rows, which I finish before release.
- **Blocks.** Which catalog entries the first release loads, and which services are P0 (item 3).

Decided Oct 19, 2026. AWS ships first.

2. Cost in the first provider comparison

- **What it is.** F11 (cost estimate from instance and pricing inputs) is P1. F18 compares one topology across providers. The question is whether the first comparison also shows cost.
- **Options.** (a) No: the first comparison covers capacity and limits only. (b) Yes: every catalog entry also needs a sourced price with a read date and a freshness rule.
- **Recommendation.** (a). Prices change often and need their own sourcing rules. The first release's strength is sourced capacity, so add cost afterward.
- **Blocks.** Whether catalog records need a price field now, and the size of the catalog phase.

Decided Oct 19, 2026. (a): the first comparison covers capacity and limits only; cost comes afterward.

3. Which services are P0

- **What it is.** P0 means the service must exist in the first catalog. Verified limits covers 18 services at priority 1 (catalog statuses: see Components by provider).
- **Options.** Keep all 18. Shrink to what the reference topology needs (load balancer, app compute, cache, relational database, queue and worker). Or add priority 2 services.
- **Recommendation.** Treat the 18 as the ceiling for the first provider, and build in the order the reference topology needs so the first demo does not wait for all 18. A service with an engine-capping limit still marked Unknown does not ship until that row is resolved.
- **Blocks.** Catalog build order, and how many Unknown and Partial rows I must close first.

Decided Oct 19, 2026. The 18 are the ceiling, built in reference-topology order; a service with an engine-capping limit still Unknown does not ship.

4. Pub/Sub as one entry under Queue and Event bus

- **What it is.** Google Pub/Sub works as a queue (pull) and as an event bus (push, fan-out). The catalog has the kinds Queue and Event bus.
- **Options.** (a) One entry listed under both kinds, with its configuration choosing pull or push. (b) Two entries.
- **Recommendation.** (a). The limits are the same service's limits, so two entries would duplicate records and drift apart. The cost is that the kind model must let one service sit in two kinds.
- **Blocks.** The canonical component schema (System model), and how GCP is shown.

Decided Oct 19, 2026. (a): Pub/Sub is one entry listed under both kinds.

5. Engine language

- **What it is.** The simulation engine runs in a Web Worker so the canvas stays responsive (N4). The question is what it is written in.
- **Options.** TypeScript: one language across UI, engine and tests, and the Engine interfaces types already compile; slower on very large discrete-event runs. Rust compiled to WebAssembly: faster for large runs, but two languages, a slower start and harder debugging.
- **Recommendation.** TypeScript first, behind a stable engine interface, and measure it in Phase 1. If the discrete-event limits (item 8) turn out too tight, the core can be swapped to Rust/WASM later without touching the UI.
- **Blocks.** Starting Phase 1.

Decided Oct 19, 2026. TypeScript first behind a stable engine interface; measure in Phase 1, swap the core to Rust/WASM later only if needed.

Group 2. Validation and quality

6. Validation data

- **What it is.** Fidelity needs a real system to compare against. Public datasets have latency and, for the Alibaba trace, container CPU and memory, but no saturation ground truth. They describe microservice meshes, not databases, caches or queues. Licenses are unchecked.
- **Options.** (a) Public data only, with a narrow claim: the simulated latency-versus-load curve of an app service stays within a stated error. (b) Public data plus one small reference test for the bottleneck check. Reference topology: load balancer, app

service, cache, Postgres, queue and worker pool.

- **Recommendation.** (b). It supports the full goal at small cost. You chose public data on Oct 19, so (a) is the fallback if you do not want to run a test.
- **Blocks.** Item 7, and the first Measured profiles.

Decided Oct 19, 2026. (b): public data plus one small reference test. Until that test is run, no profile is labeled Measured.

7. How fidelity is measured

- **What it is.** N1 has no number now (the Blueprint's 92% is dropped). It still needs a definition of "accurate".
- **Options.** A: the simulator names the same first-saturating component as the real test, as a match rate over scenarios. B: the predicted load at which latency climbs is within a stated error of the measured load. C: rank correlation of the order in which components saturate.
- **Recommendation.** A and B as the pass checks, C as a diagnostic only. Choose the tolerance for B after the first real measurement. With public data only (item 6a), neither A nor B can be computed.
- **Blocks.** N1 wording, and the Results screen's fidelity column.

Decided Oct 19, 2026. A and B are the pass checks, C is a diagnostic. The tolerance for B is set after the first real measurement.

8. Numeric limits (N6)

- **What it is.** Maximum nodes per canvas, simulated duration, supported browsers and discrete-event request volume. N4 asks for 60 FPS and a responsive UI.
- **Options.** Set numbers now by estimate, or set them after measuring a Phase 1 prototype.
- **Recommendation.** Measure first. Until then the app makes no limit claims. For browsers, plan on current desktop Chrome, Firefox and Safari.
- **Blocks.** The engine language check (item 5), and what the app tells a user who goes past a limit.

Decided Oct 19, 2026. Measure first; no limit claims until then. Browsers: current desktop Chrome, Firefox and Safari.

9. Test tolerances

- **What it is.** Tests J1 to J12 and N7 compare numbers. They need a stated tolerance for "equal".
- **Options.** Tight tolerances where the answer is exact (for example the M/M/c example, 16 servers at 150 per second giving 93.75% utilization and about 73 ms). Looser tolerances where fluid-flow and discrete-event modes are expected to differ.
- **Recommendation.** Exact-formula tests get a very tight tolerance. Cross-mode tests start with a looser starting guess (5% on utilization and latency, labeled a guess) and tighten once we see real differences.
- **Blocks.** Writing J1 to J4 for Phase 1.

Decided Oct 19, 2026. Exact-formula tests get a very tight tolerance; cross-mode tests start at 5% (a guess, labeled) and tighten later.

10. The circuit-breaker rule

- **What it is.** Validation and tests defines the breaker states. The rule is decided: it opens on the error rate over a sliding window with a minimum request count.
- **Options.** Error rate over a sliding window with a minimum request count; consecutive failures; or either one. Then how many probe requests the half-open state allows.
- **Recommendation.** Error rate over a sliding window with a minimum request count, plus a fixed number of half-open probes. All parameters stay configurable with documented defaults.
- **Blocks.** The edge contract (F19) and the retry-storm tests.

Decided Oct 19, 2026. Error rate over a sliding window with a minimum request count, plus a fixed number of half-open probes; all parameters configurable with documented defaults.

Group 3. Calibration and catalog rules (proposed, not confirmed)

11. Status "Partial" and the extra limit types

- **What it is.** The verification found values that do not fit the first limit types: documented but adjustability not checked, or a setting rather than a ceiling.
- **Options.** Add the status Partial and the types configurable setting, documented behavior, and quota with adjustability not checked. Or fold them into Unknown.

- **Recommendation.** Add them. The engine rule stays the same: only a hard limit or an adjustable quota may cap a component, so the new types can never silently limit a simulation.
- **Blocks.** F26 and F27 wording, and the Catalog screen's chips.

Decided Oct 19, 2026. Add the status Partial and the extra limit types; only a hard limit or an adjustable quota may cap a component.

12. Conditions and retirement

- **What it is.** One service can have different limits by variant: for example 65,000 clients for ElastiCache serverless caches while node-based clusters are Unknown. Variants also retire: Azure Cache for Redis Basic, Standard and Premium retire on September 30, 2028.
- **Options.** Conditions as structured fields the engine can match, or as free text. A retirement status and date per variant, or none.
- **Recommendation.** Structured conditions, so the engine picks the value that matches the user's configuration, plus a retirement status and date per variant shown on the Catalog screen.
- **Blocks.** The limit-record type in Catalog value rules, and the configuration step of the three-level component choice.

Decided Oct 19, 2026. Structured conditions the engine can match, plus a retirement status and date per variant.

13. Typed values (F28): is a source link checked

- **What it is.** A user who types values and adds a link earns the label Vendor figure. A link proves nothing by itself.
- **Options.** No check; a format check; a reachability check; or a human review.
- **Recommendation.** Accept any valid link and record the date, but show the label as Vendor figure with "source not checked" until someone confirms it. This keeps the label honest.
- **Blocks.** The wording of the confidence labels on the Calibration screen.

Decided Oct 19, 2026. Accept any valid link and record the date; show Vendor figure with "source not checked" until confirmed.

14. Load-test CSV: columns and mapping

- **What it is.** The minimum columns are timestamp, request class, latency in ms, status, and concurrency or requests per second. Tools name their columns differently.
- **Options.** A strict header the user must match; a mapping screen; or a mapping screen with presets for common tools.
- **Recommendation.** A mapping screen with presets for JMeter and k6, with the strict header as the default if the file already matches.
- **Blocks.** The import flow on the Calibration screen.

Decided Oct 19, 2026. A mapping screen with presets for JMeter and k6; the strict header is the default when the file already matches.

15. How old a profile or quota may be

- **What it is.** Provider limits and measurements age. The plan says profiles show their age and are flagged after a set time.
- **Options.** One fixed age; an age per source type; or flag when the provider page changes (needs monitoring).
- **Recommendation.** One default age (6 months is a starting guess), adjustable per profile, with no automatic monitoring in the first release.
- **Blocks.** The freshness flag on the Calibration and Catalog screens.

Decided Oct 19, 2026. One default age (6 months, a starting guess), adjustable per profile; no automatic monitoring in the first release.

16. How to fit service times from a CSV

- **What it is.** The import must turn raw latencies into something the engine can sample.
- **Options.** A statistics package that fits a distribution; our own small fit; or no fit, sampling directly from the measured values.
- **Recommendation.** No fit in the first release. Show median and P95 from the data and let discrete-event mode resample the measured values, which avoids guessing a distribution shape. Low-sample files get the low-sample label. Add a fitted distribution later if the engine needs one.
- **Blocks.** The engine's service-time model, and Phase 1 scope.

Decided Oct 19, 2026. No fit in the first release: show median and P95 and resample measured values; low-sample files get the low-sample label.

17. Cross-provider connections

- **What it is.** A design can now span providers. A connection between components on different providers crosses the internet or a private link and has its own latency, bandwidth and charges. The catalog describes services, not links between providers.
- **Options.** (a) A normal edge with user-entered latency and capacity, labeled Estimated, and no catalog values. (b) A link catalog with sourced values for common provider pairs (egress price, documented bandwidth), with latency still unmeasured. (c) No direct cross-provider edges: the user must place a modeled gateway component.
- **Recommendation.** (a) first, plus a link profile in the calibration library so a measured value can replace the estimate. Add (b) later. The Workspace example already works this way: the Azure to AWS copy link shows its capacity as Estimated.
- **Blocks.** The edge contract (F19, Shared edge contract), which needs a link capacity field, and the cost estimate (item 2), because egress is charged.

Decided Oct 19, 2026. (a): a normal edge with user-entered latency and capacity, labeled Estimated, plus a link profile in the calibration library; a link catalog comes later.

Changelog

Date	Change
Oct 20, 2026	Catalog audit applied. "Components by provider" rewritten: one table per provider with a verify link per service (read Oct 20, 2026), the limits-on-page value and four statuses (Verified, Partial, Listed (not checked), No limits on page) replacing the "Limits checked" column. 17 services that were marked checked without a recorded reading were reclassified (13 to Listed (not checked); Azure Front Door, Cloud CDN, Apigee to Partial; Azure Content Delivery Network to No limits on page), plus S3 and Azure Load Balancer to Partial. Pub/Sub link fixed to https://docs.cloud.google.com/pubsub/quotas. Dataproc noted as renamed to Managed Service for Apache Spark (new quotas link). Kubernetes cluster size limits get their own row (Kubernetes scalability thresholds, Partial). Naming: "Application Load Balancer" (ALB only as a short label); canvas "Viewers" and "Uploaders" are the Generic Clients component; ECR is not a catalog entry. Counts: 137 services, 18 Verified, 6 Partial, 104 Listed (not checked), 9 No limits on page. Provider catalogs intro, Verified limits intro and Open decision on P0 services updated to match.